

RESOLUÇÃO DE PROBLEMAS COM GRAFOS

Problema H

Kefa and Park

GRUPO C

CARLOS ANDREY, 2420515
JULIANA MARTINS, 2420516
PEDRO HENRIQUE, 2420509

Problema

Kefa decidiu comemorar seu primeiro grande salário indo ao restaurante. Ele mora perto de um parque incomum. O parque é uma **árvore enraizada** contendo **n vértices**, com a raiz no **vértice 1** (casa de Kefa).

Os vértices das folhas do parque contêm restaurantes. Kefa quer escolher um restaurante para onde irá, mas infelizmente ele tem muito medo de gatos, então não há como ele ir ao restaurante se o caminho do restaurante para sua casa contiver mais de **m vértices** consecutivos com gatos.

Sua tarefa é ajudar Kefa a contar o número de restaurantes que ele pode ir.

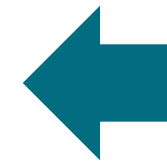
Aumento do limite máximo de chamadas recursivas, como $2 \leq n \leq 10^5$, em um cenário do pior caso (onde árvore é uma linha reta), a DFS faria até 100.000 chamadas profundas.

Lê a primeira linha da entrada e separa em duas variáveis:
n: número total de vértices da árvore.
m: quantidade máxima de gatos consecutivos ($1 \leq m \leq n$).

Lê o vetor de **presença de gatos (0 ou 1)** e insere valor 0 na posição zero da lista.
indexação 1-based: os vértices vão de **1 a N**, o zero no início faz o vértice 1 ficar no índice **cats[1]** e assim por diante

```
1 import sys
2 sys.setrecursionlimit(200000)
3
4 n, m = map(int, input().split())
5
6 cats = [0] + list(map(int, input().split()))
```

```
8  adj = [[] for _ in range(n + 1)]
9
10 for _ in range(n - 1):
11     u, v = map(int, input().split())
12     adj[u].append(v)
13     adj[v].append(u)
14
15 ans = 0
```

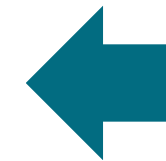


Cria uma lista de listas com tamanho **$N + 1$** vazias.

Executa um laço **$N - 1$** vezes para ler cada aresta **(u, v)** , adiciona **v** na lista de **u** e **u** na lista de **v** , garantindo que a conexão seja **bidirecional**.

Inicializa a variável acumuladora **ans** (**answer**), que armazenará o número **total de restaurantes válidos** encontrados.

```
17 def dfs(v, pai, consecutivos):
18     global ans
19
20     if cats[v]:
21         consecutivos += 1
22     else:
23         consecutivos = 0
24
25     if consecutivos > m:
26         return
27
28     folha = True
29
30     for viz in adj[v]:
31         if viz != pai:
32             folha = False
33             dfs(viz, v, consecutivos)
34
35     if folha:
36         ans += 1
```

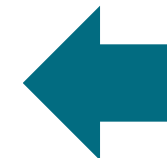


v: vértice atual sendo visitado.
pai: vértice de onde a chamada veio (evita que a busca volte para trás).
consecutivos: contagem acumulada de gatos consecutivos até o nó anterior.

Atualização da Contagem: se o nó atual **cats[v]** tem gato (**1**), incrementa consecutivos, se não tem (**0**), reseta o contador para 0.

Se **consecutivos > m**, a função é interrompida com **return**, a busca para imediatamente e descarta toda a subárvore que depende desse vértice.

```
17 def dfs(v, pai, consecutivos):
18     global ans
19
20     if cats[v]:
21         consecutivos += 1
22     else:
23         consecutivos = 0
24
25     if consecutivos > m:
26         return
27
28     folha = True
29
30     for viz in adj[v]:
31         if viz != pai:
32             folha = False
33             dfs(viz, v, consecutivos)
34
35     if folha:
36         ans += 1
```

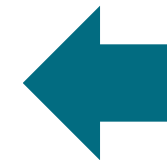


variável folha: inicializa como **True**. Se o nó possuir ao menos um filho (**viz != pai**), essa variável muda para **False**.

Para cada vizinho (que não seja o pai), chama a **dfs()** recursivamente passando o filho, o vértice atual como novo pai, e a contagem de gatos atualizada.

Se após olhar todos os vizinhos a variável **folha** continuar **True**, significa que o vértice é um **restaurante**. Incrementa **ans += 1**.

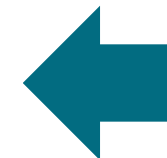
```
38     dfs(1, 0, 0)
39
40     print(ans)
```



Inicia a travessia a partir da **raiz 1**, definindo seu **pai** como **0** (**inexistente**) e a contagem inicial de **gatos consecutivos** como **0**.

Imprime o total acumulado em **ans** após a conclusão da **DFS**.

```
5 1
1 0 1 0 1
1 2
1 3
2 4
2 5
2
```



Para o caminho **1 → 3**:
O vértice **1** tem gato (**consecutivos = 1**).
O vértice **3** tem gato (**consecutivos = 2**).
Como **2 > 1**, a função aciona o **return** e o restaurante **3** não é contado.

Para os caminhos **1 → 2 → 4** e **1 → 2 → 5**:
O vértice **2** não tem gato (**consecutivos = 0**),
resetando a contagem. Chega às folhas **4** e **5**
com **consecutivos = 0**, incrementando **ans**
duas vezes.

Saída final: 2
(total de restaurantes
válidos encontrados)

RESOLUÇÃO DE PROBLEMAS COM GRAFOS

OBRIGADO!

GRUPO C

CARLOS ANDREY, 2420515
JULIANA MARTINS, 2420516
PEDRO HENRIQUE, 2420509